



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA DEPARTAMENTO DE
SISTEMAS Y COMPUTACIÓN

SEMESTRE 9

Enero - Junio 2026

Ingeniería en Sistemas Computacionales

Patrones de diseño

Documentación-Patrón de Diseño Estado

Diaz Morales Katherine Giselle 22210302

Maribel Guerrero Luis

Tijuana Baja California, a 06 de mayo del 2026

¿Qué es el patrón de diseño Estado (State)?

Es un patrón de comportamiento que permite que un objeto cambie su comportamiento cuando cambia su estado interno, como si el objeto modificara su clase en tiempo de ejecución. En lugar de utilizar múltiples estructuras condicionales como if o switch, este patrón encapsula cada comportamiento en clases separadas llamadas estados, las cuales implementan una misma interfaz. De esta forma, el objeto principal (contexto) delega su comportamiento al estado actual, logrando un código más organizado, flexible y fácil de mantener.

El patrón se utiliza cuando un objeto puede encontrarse en diferentes estados y cada uno de ellos implica un comportamiento distinto. Esto evita tener código complejo y repetitivo, permitiendo que cada estado sea independiente y reutilizable.

Práctica

La práctica consiste en el desarrollo de una simulación de un vehículo Waymo utilizando el patrón de diseño Estado. En este sistema, el vehículo cambia su comportamiento dependiendo de distintas situaciones del entorno, como la detección de obstáculos, semáforos, peatones, nivel de batería y llegada al destino. Se implementaron múltiples estados como Encendido, En Ruta, Detectando Obstáculo, Cambio de Carril, Frenando, Semáforo, Peatón, Batería, Destino y Apagado. Cada uno de estos estados define cómo debe comportarse el vehículo en ese momento, por ejemplo, reducir la velocidad, detenerse, cambiar de carril o continuar su recorrido.

Código

Carro

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WaymoPatronDiseñoEstado
{
    public class Carro
    {
        private IEstadoCarro estado;
        public void SetEstado(IEstadoCarro estado)
        {
            this.estado = estado;
        }
        public void Manejar(Form1 form)
        {
            estado.Manejar(this, form);
        }
    }
}
```

IEstadoCarro

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WaymoPatronDiseñoEstado
{
    public interface IEstadoCarro
    {
        void Manejar(Carro carro, Form1 form);
    }
}
```

AceptaViaje

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WaymoPatronDiseñoEstado
{
    public class AceptaViaje : IEstadoCarro
    {
        public void Manejar(Carro carro, Form1 form)
        {
            form.ActualizarEstado("Viaje aceptado");
        }
    }
}
```

```
}  
}
```

Alto

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
  
namespace WaymoPatronDiseñoEstado  
{  
    public class Alto : IEstadoCarro  
    {  
        public void Manejar(Carro carro, Form1 form)  
        {  
            form.velocidad = 2;  
            form.ActualizarEstado("Alto (detenido)");  
        }  
    }  
}
```

Apagado

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
  
namespace WaymoPatronDiseñoEstado  
{  
    public class Apagado : IEstadoCarro  
    {  
        public void Manejar(Carro carro, Form1 form)  
        {  
            form.velocidad = 0;  
            form.ActualizarEstado("Carro apagado");  
        }  
    }  
}
```

Bateria

```
using System;  
using System.Collections.Generic;  
using System.Linq;  
using System.Text;  
using System.Threading.Tasks;  
  
namespace WaymoPatronDiseñoEstado  
{  
    public class Bateria : IEstadoCarro  
    {  
        public void Manejar(Carro carro, Form1 form)
```

```

        {

            form.velocidad = 1;
            form.ActualizarEstado("Bateria baja");

        }
    }
}

```

CambioCarril

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WaymoPatronDiseñoEstado
{
    public class CambioCarril : IEstadoCarro
    {
        public void Manejar(Carro carro, Form1 form)
        {
            form.ActualizarEstado("Cambiando de carril");

            form.CambiarCarril();
            // Después de cambiar, vuelve a ruta
            carro.SetEstado(new EnRuta());
        }
    }
}

```

Carro

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WaymoPatronDiseñoEstado
{
    public class Carro
    {
        private IEstadoCarro estado;
        public void SetEstado(IEstadoCarro estado)
        {
            this.estado = estado;
        }
        public void Manejar(Form1 form)
        {
            estado.Manejar(this, form);
        }
    }
}

```

Destino

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WaymoPatronDiseñoEstado
{
    public class Destino : IEstadoCarro
    {
        public void Manejar(Carro carro, Form1 form)
        {
            form.velocidad = 0;
            form.ActualizarEstado("Destino alcanzado");
        }
    }
}
```

DetectandoObstaculo

```
using System;
using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WaymoPatronDiseñoEstado
{
    public class DetectandoObstaculo : IEstadoCarro
    {
        public void Manejar(Carro carro, Form1 form)
        {
            form.velocidad = 2;
            form.ActualizarEstado("Detectando obstáculo");

            // Cambia automáticamente a frenando
            carro.SetEstado(new Frenando());
        }
    }
}
```

Encendido

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WaymoPatronDiseñoEstado
{
    public class Encendido : IEstadoCarro
```

```

    {
public void Manejar(Carro carro, Form1 form)
    {
        form.ActualizarEstado("Carro encendido");
    }
    }
}

```

EnRuta

```

using System;
using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WaymoPatronDiseñoEstado
{
    public class EnRuta : IEstadoCarro
    {
public void Manejar(Carro carro, Form1 form)
    {
        form.velocidad = 5;
        form.ActualizarEstado("Carro en ruta");
        form.BackColor = Color.FromArgb(18, 80, 40);
    }
    }
}

```

Frenando

```

using System;
using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace WaymoPatronDiseñoEstado
{
    public class Frenando : IEstadoCarro
    {
public void Manejar(Carro carro, Form1 form)
    {
        form.velocidad = 0;
        form.ActualizarEstado("Frenando: " + form.motivoFreno);
        form.BackColor = Color.FromArgb(120, 30, 30);
    }
    }
}

```

Peaton

```

using System;
using System.Collections.Generic;

```

```

        using System.Linq;
        using System.Text;
        using System.Threading.Tasks;

        namespace WaymoPatronDiseñoEstado
        {
            public class Peaton : IEstadoCarro
            {
                public void Manejar(Carro carro, Form1 form)
                {
                    form.velocidad = 0;
                    form.ActualizarEstado("Peaton cruzando - detenido");
                }
            }
        }

```

Semaforo

```

        using System;
        using System.Collections.Generic;
        using System.Linq;
        using System.Text;
        using System.Threading.Tasks;

        namespace WaymoPatronDiseñoEstado
        {
            public class Semaforo : IEstadoCarro
            {
                public void Manejar(Carro carro, Form1 form)
                {
                    form.velocidad = 0;
                    form.ActualizarEstado("Semaforo en rojo - Detenido");
                }
            }
        }

```

Form1

```

        using System;
        using System.Collections.Generic;
        using System.ComponentModel;
        using System.Data;
        using System.Drawing;
        using System.Linq;
        using System.Text;
        using System.Threading.Tasks;
        using System.Windows.Forms;

        namespace WaymoPatronDiseñoEstado
        {
            public partial class Form1 : Form
            {
                private Carro carro = new Carro();
                public int velocidad = 0;
                int bateria = 100;
                int contadorBateria = 0;
                bool semaforoRojo = true;
                int contadorSemaforo = 0;
            }
        }

```

```

        bool peatonCruzando = false;
        bool peatonYaCruzo = false;

        int posicionFinalPeaton;
        int posicionInicialPeaton;
        int carrilOriginal;
        List<string> historialEstados = new List<string>();
        public string motivoFreno = "";
        public Form1()
        {
            InitializeComponent();

            carrilOriginal = picCarro.Top;
            // Estado inicial
            carro.SetEstado(new Apagado());
            carro.Manejar(this);

            posicionInicialPeaton = picPeaton.Top;
            posicionFinalPeaton = picPeaton.Top + 300;

            // Fondo general
            this.BackColor = Color.FromArgb(18, 80, 40);

            // Carretera
            panelRoad.BackColor = Color.FromArgb(50, 50, 50);

            // Texto principal
            lblEstado.ForeColor = Color.White;
            lblEstado.Font = new Font("Segoe UI", 16, FontStyle.Bold);

            // Bateria
            lblBateria.ForeColor = Color.LightGreen;
            lblBateria.Font = new Font("Segoe UI", 10, FontStyle.Bold);

            // Botones
            btnIniciar.BackColor = Color.FromArgb(0, 120, 215);
            btnIniciar.ForeColor = Color.White;
            btnIniciar.FlatStyle = FlatStyle.Flat;

            btnReiniciar.BackColor = Color.FromArgb(80, 80, 80);
            btnReiniciar.ForeColor = Color.White;
            btnReiniciar.FlatStyle = FlatStyle.Flat;

            DibujarLineasCarril();
        }
        private void DibujarLineasCarril()
        {
            for (int i = 0; i < panelRoad.Width; i += 80)
            {
                Panel linea = new Panel();
                linea.BackColor = Color.Gold;
                linea.Width = 40;
                linea.Height = 5;
                linea.Top = panelRoad.Height / 2;
                linea.Left = i;
            }
        }

```

```

        panelRoad.Controls.Add(linea);
    }
}

public void ActualizarEstado(string mensaje)
{
    lblEstado.Text = mensaje;
    if (historialEstados.Count == 0 || historialEstados.Last() != mensaje)
    {
        historialEstados.Add(mensaje);
    }
}

public void CambiarCarril()
{
    picCarro.Top -= 15;

    // limite
    if (picCarro.Top < carrilOriginal - 40)
    {
        picCarro.Top = carrilOriginal - 40;
    }
}

public void MostrarHistorial()
{
    string historial = "Historial de estados:\n\n";

    foreach (string estado in historialEstados)
    {
        historial += "- " + estado + "\n";
    }

    MessageBox.Show(historial, "Recorrido finalizado");
}

private void btnEncender_Click(object sender, EventArgs e)
{
    carro.SetEstado(new Encendido());
    carro.Manejar(this);
}

private void btnRuta_Click(object sender, EventArgs e)
{
    carro.SetEstado(new EnRuta());
    carro.Manejar(this);
}

private void btnObstaculo_Click(object sender, EventArgs e)
{
    carro.SetEstado(new DetectandoObstaculo());
    carro.Manejar(this);
}

private void btnFrenar_Click(object sender, EventArgs e)
{
    carro.SetEstado(new Frenando());
    carro.Manejar(this);
}

```

```

private void panelCarretera_Paint(object sender, PaintEventArgs e)
{

}

private void btnApagado_Click(object sender, EventArgs e)
{
    carro.SetEstado(new Apagado());
    carro.Manejar(this);
}

private void lblEstado_Click(object sender, EventArgs e)
{

}

private void btnIniciar_Click(object sender, EventArgs e)
{
    carro.SetEstado(new AceptaViaje());
    carro.Manejar(this);

    carro.SetEstado(new Encendido());
    carro.Manejar(this);

    carro.SetEstado(new EnRuta());
    carro.Manejar(this);

    timer1.Start();
}

private void timer1_Tick(object sender, EventArgs e)
{
    // Movimiento del carro
    picCarro.Left += velocidad;

    // Distancias
    int distancia = picObstaculo.Left - picCarro.Right;
    int distanciaAlto = picAlto.Left - picCarro.Right;
    int distSemaforo = picSemaforo.Left - picCarro.Right;
    int distDestino = picDestino.Left - picCarro.Right;
    int distPeaton = picPeaton.Left - picCarro.Right;

    // BATERÍA
    contadorBateria++;

    if (contadorBateria >= 10)
    {
        bateria--;
        contadorBateria = 0;
    }

    lblBateria.Text = "Batería: " + bateria + "%";
}

```

```

        // SEMAFORO
        contadorSemaforo++;

        if (contadorSemaforo >= 30)
        {
            semaforoRojo = !semaforoRojo;
            contadorSemaforo = 0;

            if (semaforoRojo)
                picSemaforo.BackColor = Color.Red;
            else
                picSemaforo.BackColor = Color.LimeGreen;
        }

        // PEATON CRUZANDO
        if (peatonCruzando && !peatonYaCruzo)
        {
            picPeaton.Top += 3;

            if (picPeaton.Top >= posicionFinalPeaton)
            {
                picPeaton.Top = posicionFinalPeaton;
                peatonCruzando = false;
                peatonYaCruzo = true;
            }
        }

        // DESTINO
        if (distDestino < 50 && distDestino > 0)
        {
            carro.SetEstado(new Destino());
            carro.Manejar(this);
            timer1.Stop();
            MostrarHistorial();
            return;
        }

        // BATERÍA AGOTADA
        if (bateria <= 0)
        {
            carro.SetEstado(new Frenando());
            carro.Manejar(this);

            carro.SetEstado(new Apagado());
            carro.Manejar(this);

            timer1.Stop();

            MostrarHistorial();
            return;
        }
        if (contadorBateria >= 10)
        {
            bateria--;

            if (bateria < 0)
                bateria = 0;
        }
    
```

```

        contadorBateria = 0;
    }

    // PEATÓN
    if (distPeaton < 40 && distPeaton > 0 && !peatonYaCruzo)
    {
        peatonCruzando = true;

        motivoFreno = "Peaton cruzando";
        carro.SetEstado(new Frenando());
        carro.Manejar(this);
        return;
    }

    // semaforoo
    if (distSemaforo < 100 && distSemaforo > 0)
    {
        if (semaforoRojo)
        {
            motivoFreno = "Semaforo en rojo";
            carro.SetEstado(new Frenando());
        }
        else
        {
            carro.SetEstado(new EnRuta());
        }

        carro.Manejar(this);
        return;
    }

    // ALTO
    if (distanciaAlto < 100 && distanciaAlto > 0)
    {
        carro.SetEstado(new Alto());
        carro.Manejar(this);
        return;
    }

    // OBSTÁCULO LEJOS → SOLO DETECTA
    if (distancia < 120 && distancia > 60)
    {
        carro.SetEstado(new DetectandoObstaculo());
        carro.Manejar(this);
        return;
    }

    // OBSTÁCULO CERCA → CAMBIA DE CARRIL
    if (distancia <= 60 && distancia > 0)
    {
        carro.SetEstado(new CambioCarril());
        carro.Manejar(this);
        return;
    }

    // OBSTÁCULO MUY CERCA → FRENAR
    if (distancia <= 30 && distancia > 0)
    {
        motivoFreno = "Obstaculo";
    }

```

```

        carro.SetEstado(new Frenando());
        carro.Manejar(this);
        return;
    }

    // NORMAL
    carro.SetEstado(new EnRuta());
    carro.Manejar(this);
}

private void picCarro_Click(object sender, EventArgs e)
{
    picCarro.Top -= 40;
    picCarro.BackColor = Color.Transparent;

}

private void picObstaculo_Click(object sender, EventArgs e)
{

}

private void pictureBox1_Click(object sender, EventArgs e)
{

}

private void ReiniciarSimulacion()
{
    picCarro.Left = 40;
    velocidad = 0;

    carro.SetEstado(new Apagado());
    carro.Manejar(this);
}

private void btnReiniciar_Click(object sender, EventArgs e)
{
    timer1.Stop();
    ReiniciarSimulacion();
}

private void pictureBox1_Click_1(object sender, EventArgs e)
{

}

private void lblBateria_Click(object sender, EventArgs e)
{
    lblBateria.Text = "Batería: 100%";
}

private void btnPeaton_Click(object sender, EventArgs e)
{
    carro.SetEstado(new Peaton());
    carro.Manejar(this);
}
}

```

Carro apagado



Frenado



Alto



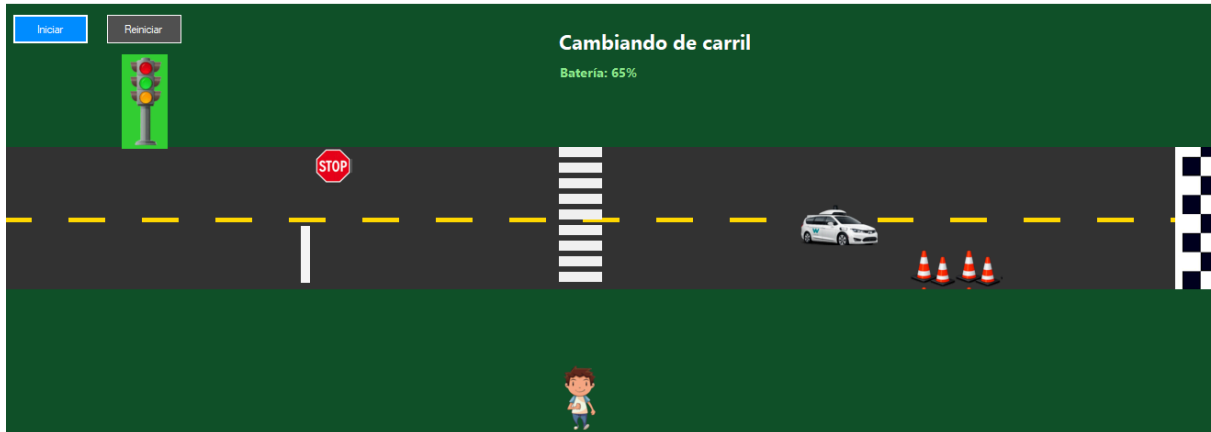
Peatón cruzando



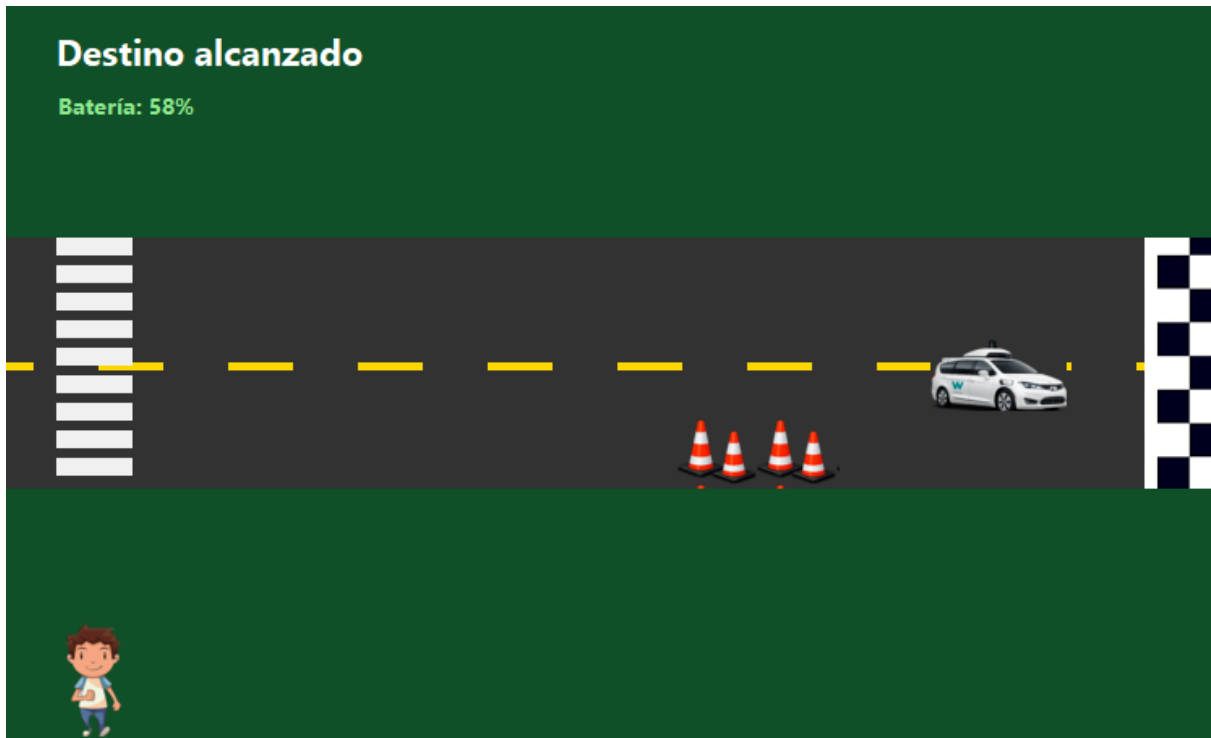
Detecta obstáculo



Cambia de carril



Destino alcanzado



Historial de recorrido

Recorrido finalizado



Historial de estados:

- Carro apagado
- Viaje aceptado
- Carro encendido
- Carro en ruta
- Frenando: Semáforo en rojo
- Carro en ruta
- Alto (detenido)
- Carro en ruta
- Frenando: Peaton cruzando
- Carro en ruta
- Detectando obstáculo
- Cambiando de carril
- Carro en ruta
- Destino alcanzado

Aceptar

Batería en 0, el carro se apaga



Conclusión

El patrón State es una herramienta fundamental en sistemas como simuladores o videojuegos, en donde los cambios de estado son constantes, por ejemplo en esta práctica me ayudó mucho ya que se pudo observar cómo es que este patrón es útil en sistemas donde existen múltiples condiciones y cambios de comportamiento, además al separar cada estado en clases independientes y se logra un código más limpio y fácil de entender